Enhanced Spectral-line Pipeline for the ALMA Data Archive (ESPADA)

Author: Borja Montoro Molina

Version: 1.0

2026

Contents

1 Introduction

The ability to provide science-ready advanced data products advanced data product (ADP) from a completed observation to the scientific community is the ultimate goal of most modern observatories. ADPs provide a myriad of advantages that encourage scientific discoveries, stimulate the participation of non-facility-specific experts, and curate their development into future users. ADPs facilitate the ability to carry out multi-wavelength counterpart identification; categorization of objects through their morphological and spectral properties; and rapid recognition of serendipitous discoveries. These advantages provide both short and long term benefits to researchers focused on individual objects, or on large populations of sources alike while increasing the legacy value of an observatory and its archive.

As the most powerful sub/millimeter telescope in the world, with a sophisticated science calibration and imaging pipeline, ALMA is a natural target for an ADP pipeline. In the past, the generation of ADPs has often been labor intensive and tailored to specific science goals for the best results, but algorithmic advances in source finding, and recent pushes to prepare for large surveys which will dominate observing time on many future radio telescopes have led to significant development and innovation. In particular, recent developments are being led by neutral hydrogen (HI) spectral line surveys at and below 1.42 GHz (e.g. Apertif, van Cappellen et al. 2022; Adams et al. 2022; Hess et al. in prep; and WALLABY, Westmeier et al. 2022).

Enhanced Spectral-line Pipeline for the ALMA Data Archive (ESPADA) was designed to address his need. It is an end-to-end python-based pipeline which acts as a wrapper and control package for the downloading of ALMA data from the science archive, and the subsequent source finding, parameterization, and visualization workflow, using the existing Source Finding Application (SoFiA)[1] and SoFiA Imaging Pipeline (SIP)[2] packages. The former is a robust and well developed 3D source finding package, which derives source parameters; and the the latter takes source catalogs and FITS files from SoFiA and generates inspection plots of the individual sources. ESPADA is designed to generate advanced ALMA spectral line data products with minimal user intervention. However, it is also built with the flexibility to accommodate user customization of the workflow and source finding for a variety of use cases. It should be noted upfront that in the SoFiA nomenclature, each 3D island in a data cube is considered a 'source' by SoFiA regardless of what spectral line is detected, meaning that if a single astronomical source has multiple spectral lines, each will be considered a 'source' in the SoFiA (and current ADP) nomenclature. A cube will have two 'sources' even if the emission in fact comes from the same object, but is two different spectral lines. This may be different than what has been used by others in the ALMA community, where a source may be designated as a spatial region of emission.

The backbone of the pipeline consists of three main steps. First, the user-specified data is downloaded from the ASA via its Table Access Protocol (TAP) service [3]. Second, source finding is run on the image cubes using the SoFiA source finding and parametrization software. Third, SIP is run on the results of SoFiA to generate figures for visualization of the sources that were found. The ESPADA software provides logical structure to the pipeline workflow, and generates logs and quality assurance (QA) products that report on the results of the pipeline execution.

2 ESPADA architecture overview

The ESPADA structure is modular and consists of a main module, ESPADA; and seven secondary modules: CONFIG, LOGGER, DATAP, SOPAR, SIPARGS, GROUP and REPORT. ESPADA contains the main code from which the secondary modules are called and controls the overall pipeline workflow. From

[1] https://gitlab.com/SoFiA-Admin/SoFiA-2/-/wikis/home

[2] https://github.com/kmhess/SoFiA-image-pipeline

[3] ALMA's ObsCore Table Access Protocol (TAP) service enables programmatic access to the ALMA Science Archive through Virtual Observatory (VO) standards (https://www.ivoa.net/documents/TAP/)

first to last, the secondary modules control the validation of the configuration file; the logger; data downloads from the ASA using the TAP service; source finding and parametrization (via SoFiA), and quality assurance; source visualization (via SIP), grouping of sources found in source finding; and generation of reports in JSON and HTML formats, respectively. Figure 1 provides a flowchart of ESPADA execution. Detailed descriptions of the modules are given in sections 8, 9, 10, 11, 12 and 13.

The primary ESPADA configuration file is `config.yaml`, which controls the main pipeline flow within the ESPADA module. It also specifies the paths to the parameter files required for executing the modules DATAP, SOPAR and SIPARGS which require separate parameter files. By default, these are called `download_par.yaml`, `sofia_abs_default.par`, `sofia_emi_default.par`, and `sip_args.yaml`, for DATAP, SOPAR (one for absorption and one for emission), and SIPARGS, respectively. All parameter files are detailed in Section 5, except for those for SoFiA, which are described in their own documentation. However, it is recommended to review Section 10 for parameter restrictions that apply to these. In the repository, parameters files are pre-filled with examples that are suited for ALMA data in the ASA.

The logic of the pipeline is such that if an intermediate step is disabled (for example it has been run before with satisfactory results), the pipeline can predict the input files for the next step based on the configuration file. Thus, intermediate or later steps can be rerun without re-running the entire pipeline.

3 Installation

3.1 Requirements

ESPADA relies on the SoFiA and SIP software packages; it is therefore necessary to have both installed on the system beforehand. The current version of the pipeline requires Python 3.10 or later. It is recommended to perform the installation of both the required software and the pipeline itself within an isolated virtual environment.

ESPADA makes use of the SUBPROCESS python routine to run the external software mentioned, which means that the specific command required to execute each package is needed. In the current version, the pipeline assumes that the commands to run SoFiA and SIP are `sofia` and `sofia_image_pipeline`, respectively, i.e. the default commands. It is recommended to follow the authors' installation guidelines for each package, which can be found in the documentation of their respective repositories, in order to avoid any changes to the installation.

If, for any reason, the execution commands differ from those mentioned above and it is not desired to modify the installation, parts of the pipeline code must be changed. This is an exceptional option and is not recommended. The steps to follow to make this change are available in Appendix A.

There is an additional dependency on the Ghostscript program. This is simply used to generate the final HTML report and convert some results from SoFiA-2, from .eps format to .png format. The latter has greater compatibility and can be viewed in any browser. To install it, simply use:

```
$ apt-get install ghostscript
or
$ brew install ghostscript
```

This is not a requirement, but ot is strongly recommend to install ESPADA in an isolated environment to avoid dependency conflicts. If you do not have a compatible version of Python installed, you can use `pyenv` and `pyenv-virtualenv` to manage Python versions and virtual environments.

You can install `pyenv` automatically with:

```
$ curl https://pyenv.run | bash
```

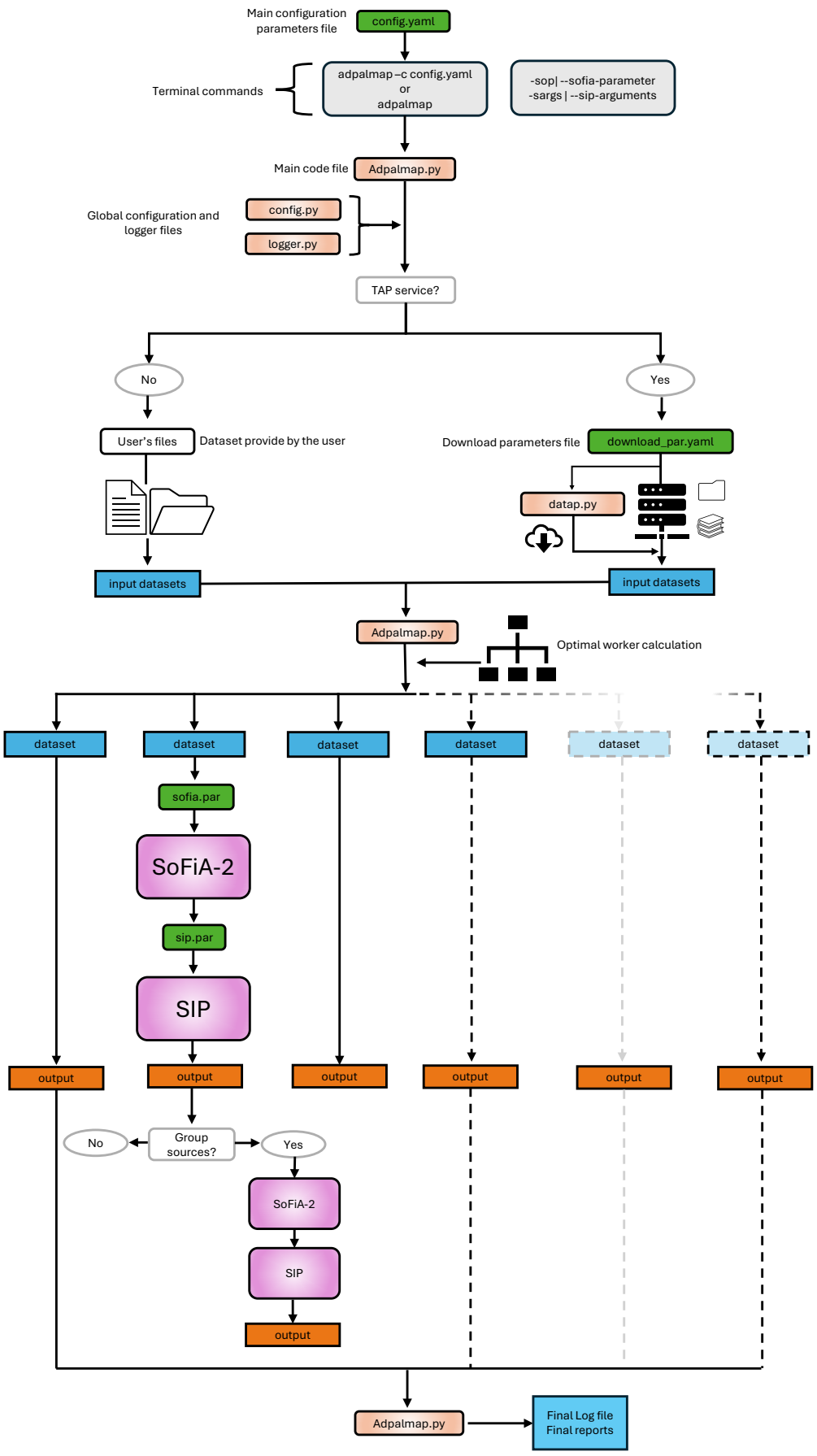

Figure 1: A general workflow diagram of espada. Green: input configuration files. Coral: python-code modules. Purple: external software. Blue: FITS files, log, and QA products. Orange: Advanced data products provided by espada (a combination of SoFiA and SIP output).

After the installation, follow the instructions displayed in the terminal to add `pyenv` to your `PATH`. Once configured, restart your terminal or reload your shell configuration:

```
$ source ~/.bashrc
```

Install Python 3.10 and create a virtual environment for ESPADA:

```
$ pyenv install 3.10
$ pyenv virtualenv 3.10 espada
$ pyenv activate espada
```

This will create an isolated virtual environment with the correct Python version. To exit the virtual environment, run:

```
$ pyenv deactivate
```

3.2 Standard installation procedure

Installing ESPADA from the GitHub repository is fairly straightforward, the user simply needs to type these commands in the terminal:

```
$ git clone https://github.com/Borjamomo96/ADP-ALMA-Pipeline.git
$ cd ADP-ALMA-Pipeline
$ pip install -e .
```

This will install the pipeline in developer mode, which is recommended but not mandatory, and will download all available files from the repository, not just the Python-related ones. Alternatively, it can be run without the "-e" option, but be aware that some files may be missing which can lead to errors.

4 Running the pipeline

The pipeline is initiated from a terminal command line with the command:

```
$ espada
```

Note that this functionality is only available if the instructions outlined in Section 1 have been properly followed. In that case, the pipeline will be installed as a command-line tool. Although the pipeline can still be executed with:

```
$ python espada
```

this approach is strongly discouraged, as it may lead to import and environment inconsistencies. From this point onward, it will be assumed that the pipeline is executed using the installed command-line interface.

The `espada` command will automatically search for the included default pipeline configuration file, `config.yaml`. However, the recommended usage is through the argument `"-c|--config-file"`:

```
$ espada -c config_path.yaml
$ espada --config-file config_path.yaml
```

where "config_path.yaml" is the user-defined configuration file, based on the default template. More details about the parameters contained in this file will be shown in the section 5.1.

4.1 Available command line arguments

In addition to the aforementioned argument "-c|--config-file", there are currently 6 other available arguments. These are explained below:

- "-cp|--config-parameters": Allows the user to modify the parameters, except for input_data_set within the config.yaml parameter file directly from the terminal. The required format is: parameter=value, with the restriction that no spaces are allowed between them. The parser will interpret any space as a new parameter and may fail. An example:

```
$ espada -c config.yaml -cp enable_tap_service=false input_file=
   Testing/espada_input_file.txt
$ espada -c config.yaml -cp num_cores=5
```

- "-sop|--sofia-parameters": Allows the user to modify any parameter within the SoFiA parameter file directly from the terminal. The required format is the same as when running SoFiA independently: parameter=value, with the restriction that no spaces are allowed between them. The parser will interpret any space as a new parameter and may fail. An example:

```
$ espada -c config.yaml -sop linker.radiusXY=2 pipeline.verbose=true
```

  Special care must be taken when using this option. ESPADA can be executed to search for emission, absorption—each with its own optimized parameter file—or both, and any parameter specified via the terminal will be applied to all SoFiA parameter files used during execution, which may lead to unintended effects. Some parameters can not be modified in order to preserve the correct functioning of the pipeline; in such cases, a warning message will indicate that the provided value will be ignored. For further details about the SOFIA module and restricted parameters, see Section 10.

- "-sarg|--sip-arguments": Similar to the previous option, this allows the user to modify any argument within the SIP parameter file directly from the terminal, with certain caveats. The general format consists of including the -sarg argument followed by the native SIP arguments: -sarg -argument value. Examples:

```
$ espada -c config.yaml --sip-arguments -i 0.15
$ espada -c config.yaml -sarg -i 0.15
$ espada -c config.yaml -sarg -i 0.15 -m -o datacube.fits
```

  It is not possible to maintain the same format for all parameters both in the parameter file and via the terminal. For example, boolean parameters in sip_args.yaml accept True|False, whereas in the terminal only the corresponding argument should be included or omitted. Since it can be confusing in some cases, minimal use of this functionality is recommended. One important aspect is that **other ESPADA arguments must not be used after this one**, as they will either be ignored or, in the worst case, the parser will attempt to parse them as parameters within -sarg, which may result in errors.

- "-h|--help": A standard default argument that displays concise documentation on the pipeline and how to execute it easily. The command for its use is:

```
$ espada -h
$ espada --help
```

- "-i|--info": Displays an additional layer of information about the configuration files and the individual parameters they contain. The format in this case depends on whether a file or a specific parameter is requested: -i file=file_name or -i parameter=parameter_name, where "file_name" and "parameter_name" can refer to any of the configuration files (using their default names) and any parameter contained within them, respectively. Some examples:

```
$ espada -i file=config.yaml
$ espada -i parameter=filename_must_include
$ espada --info parameter=make_report
```

- "−−debug": This argument changes the logging level of the messages displayed both in the terminal and in the log files, enabling additional messages as well as the traceback in case of errors. The command is:

```
$ espada -c config.yaml --debug
```

It is worth noting that ESPADA includes an additional message logging level, beyond the Python standard, called RAW, which will also be displayed. For more details, see section 7.

5 Description of the parameter files

All ESPADA parameter files use the YAML format, with the exception of the SoFiA parameter file (sopar.par), which follows its own format and rules specified in the SoFiA documentation. It is worth noting that, although SIP is executed independently via the terminal, the sipargs.yaml file adapts all of its arguments—with minor modifications to the YAML format, (see more details in 11). The structure of parameters within the parameter files must be as follows:

```
parameter :  value
```

Empty lines, as well as any sequence of characters beginning with a hash symbol (#), will be ignored and treated as comments. All parameters must be included in their corresponding parameter files and must have the correct name; otherwise, an error indicating the missing parameter will be raised. Some parameters accept blank values, in which case a default value will be assumed. However, this does not apply to all parameters, so it is strongly recommended to explicitly define a value for each of them. The default parameters provided in the SoFiA parameter files (for both emission and absorption) and in SIP are optimized to perform adequately for most datasets available in the ALMA archive. It is recommended to modify the parameters with caution and carefully evaluate the impact of the change on the results.

In the following sections, we provide a detailed explanation of each parameter file and its individual parameters.

5.1 Main pipeline configuration file - config.yaml

The config.yaml file is the main entry point and primary control file of the pipeline. It is read and processed within the CONFIG module so that all included parameters contain allowed values and there is no conflict in the workflow. The parameters are divided into six sections. These are entitled: General, Logger, Input Data, TAP Service, SoFiA, and SIP. These determine the overall flow of the pipeline that runs within the ESPADA module, setting the execution of different modules or sub-tasks and specifying the path to the necessary parameter files. The parameters defined here can be overridden dynamically via the -cp|–config-parameters terminal arguments when running the pipeline. The table below provides a description of all the parameters per block.

Table 1: Pipeline parameters

Parameter	Type	Values	Default	Description
General				
make_report	bool	True, False	False	Create a *report* directory that contains the reports in HTML and JSON formats to get a quick overview of all the results obtained in the specific run. In the case of multiple ESPADA executions, the directory names will be *report*+_%d%m%y_%H%M%S.
verbose	bool	True, False	True	Print the execution logs of SoFiA and SIP to the terminal as they are being generated.
num_cores	int	≥ 0	12	Set the maximum number of cores that can be used to run ESPADA. For more details on ESPADA parallelization, see 8.1.
output_dir	string		espada_run/	Create a directory where all products obtained during an ESPADA run will be stored. It cannot be overwritten by other directory variables.
Logger				
clear_logs	bool	True, False	True	If True, delete all existing ".log" files in the directory selected in log_file.
log_file	string		log_dir/espada.log	Path to the log file, relative to output_dir or absolute. If an absolute path is provided, it must be contained within output_dir; otherwise it will be ignored and the default path log_dir/espada.log inside output_dir will be used instead.
Input data				
input_data_set	string, dict, list			Path to the primary beam-corrected data cube file(s), the primary beam(s) (optional), and the mask(s) (optional). More details about the structure of the input data can be found in section 6.
input_file	string			Path to the text file that includes the paths to the primary beam-corrected data cube file(s), the primary beam(s) (optional), and the mask(s) (optional). More details about the structure of the input data can be found in section 6.
TAP service				

Continued on next page

Parameter	Type	Values	Default	Description
enable_tap_service	bool	True, False	True	If True, allow execution of the module DATAP, which downloads data from the ALMA archive based on parameters selected in the download_par_file.
download_par_file	string		~/ADP-ALMA-Pipeline/adplib/tap/download_par.yaml	Path to the data file that contains the parameters listed in section 5.2 below.
SoFiA				
enable_sofia	bool	True, False	True	If True, allow execution of the module SOPAR, which executes SoFiA in the mode specified in run_mode with the corresponding sofia_abs_file and sofia_emi_file files.
run_mode	string	emission, absorption, both	both	Indicate the type of emission that SoFiA should search for in the selected data cube. If *both* is selected, SoFiA will first search for absorption and then run again to search for emission.
use_pb	bool	True, False	True	If True, the primary beam file (if any) will be used in the SoFiA execution.
use_mask	bool	True, False	True	If True, the mask file (if any) will be used in the SoFiA execution.
abs_flag_cube	bool	True, False	True	If True, use the mask obtained during the first run of SoFiA in *both* mode (i.e., *absorption*) as a flag mask for emission searches. Ignored if run_mode is not *both*.
auto_setup	bool	True, False	True	If True, adjust certain SoFiA parameters based on specific keywords found in data cube headers. For more detail see the appendix B.
sofia_abs_file	string		~/ADP-ALMA-Pipeline/adplib/sofia/sofia_abs_default.par	Path to the SoFiA-2 parameter file optimized to find **absorption**.
sofia_emi_file	string		~/ADP-ALMA-Pipeline/adplib/sofia/sofia_emi_default.par	Path to the SoFiA-2 parameter file optimized to find **emission**.

Continued on next page

Parameter	Type	Values	Default	Description
SIP				
enable_sip	bool	True, False	True	If True, allow execution of the module SIPARGS, which execute SIP with the corresponding sip_par_file file.
sip_par_file	string		~/ADP-ALMA-Pipeline/adplib/sip/sip_args.yaml	Path to the SIP argument file. More details provide in section 5.3.
Group				
enable_group	bool	True, False	False	If True, allow execution of the module GROUP, which is responsible for grouping sources along the spectral axis if they have a significant spatial overlap in projection.
overlap_mode	string	absflux, flux, area	absflux	Sets the parameter used to decide whether sources should be grouped. It can be the absolute flux fraction in the overlap area, the signed flux fraction in the overlap area, or the fractional area, respectively.
overlap_threshold	int	$0 \leq x \leq 1$	0.8	Set the threshold applied to the parameter in overlap_mode. It must be between 0 and 1.

5.2 Parameter file for download - download_par.yaml

The download_par.yaml file is the dedicated configuration file for the DATAP module, which handles communication with the ASA using the TAP service. When the enable_tap_service option is activated in the main config.yaml file, ESPADA uses the parameters defined here to build Astronomical Data Query Language (ADQL) queries, execute them against the archive, and download the resulting datasets. This file is organized into three main blocks: Server (for connection settings), Query (where the search type and its parameters are defined), and download_par (for controlling the download process). The table below details all of its parameters.

Table 2: Pipeline parameters

Parameter	Type	Values	Default	Description
Server				
server_address	string	ESO URL[4], NRAO URL[5], NAOJ URL[6]	ESO URL	URL of the server website to query the data.

Continued on next page

[4] https://almascience.eso.org
[5] https://almascience.nrao.edu
[6] https://almascience.nao.ac.jp

Parameter	Type	Values	Default	Description
credentials	bool	True, False	False	If True, allow the user to log in with their ALMA Science Portal user credentials. The server may block bulk downloads for security reasons. This option allows for more permissive and, in theory, faster downloads.
stored_ credentials	bool	True, False	False	If True, allow to save the credentials between executions so that they remain stored for a certain period without having to specify them each time. Note that if this option is set to True in one execution, the credentials will be saved for a time and will persist for subsequent runs — even if the option is set to False in those later runs.
Query				
query_type	string	proposal, member_ous_id, conesearch, target, keysearch, free		ESPADA has predefined ADQL queries depending on the search type. Each query contains specific and common parameters that must be specified in query_par.
query_par:	dict			
proposal_id	string			Used when query_type : proposal. ID of a proposal available in the ALMA archive.
member_ous_id	string			Used when query_type : member_ous_id. Member_ous_id of an observation available in the ALMA archive.
ra	float			
dec	float			Used when query_type : conesearch.
search_radius	float			
sources	list			Used when query_type : target. List with the names of the different sources. Example: ['NGC4418', 'AB_Aur']
search_radius	float			
search_dict	dict			Used when query_type : keysearch. Supports a dictionary with different keywords and their values; for more information on available keywords, see the ALMINER documentation[7]. Example: {'target_name':['NGC4418'], 'proposal_id': ['2022.1.00738.S']}

Continued on next page

[7] https://alminer.readthedocs.io/en/latest/

Parameter	Type	Values	Default	Description
query_string	string			Used when query_type : free. It allows you to enter your own query using ADQL language.
Common parameter for all queries				
point	bool	True, False	False	If True, the conesearch query ignores the search_radius parameter and returns only observations whose footprint (region) exactly contains the specified coordinates (RA, Dec).
public	bool	True, False	True	If True, the query will include only public data
published	bool	True, False	False	If True, the query will include only published data
print_targets	bool	True, False	True	If True, display a message with the sources included in the search.
print_query	bool	True, False	True	If True, display a message with the query used.
download_par	dict			
data_dir	string		~ADP-ALMA-Pipeline/archive_data	Path to the directory where the downloaded data should be placed.
remove_compressed_file	bool	True, False	False	If True, remove compressed files. Such as the case for primary beam and mask cubes in the ALMA archive.
remove_archive_mask	bool	True, False	False	If True, the original float-type mask file downloaded from the ASA is deleted after being converter to integer type (required by SoFiA-2). More details in section 6.
dryrun	bool	True, False	True	If True, do a test run to check the size and number of files to download without actually downloading them.
print_urls	bool	True, False	True	If True, print to terminal the url of the files to be downloaded from the ALMA archive.
filename_must_include	list			A list of strings the user wants to be contained in the url filename.

5.3 Parameter file to control SIP - sip_args.yaml

The sip_args.yaml file is the parameter file for the SIPARGS module, which serves as an interface between ESPADA and SIP. Its purpose is to adapt all command-line arguments accepted by SIP into the YAML format, making them easier to use and configure. ESPADA reads this file to construct and execute the corresponding SIP command. Importantly, parameters defined here can be overridden dynamically via the -sarg|-sip-arguments terminal arguments when running the pipeline, offering great flexibility. The following table describes each parameter, its type, and its main function in the image and spectrum generation process.

Table 3: Pipeline parameters

Parameter	Type	Values	Default	Description
catalog_file	string, list			Catalog file obtained from the execution of SoFiA. This can be in .txt or .xml format. See section 11.1 for more details.
source_id	int, list			Allow the user to select certain sources, or range of sources from catalog based on the source id number. Default is all sources. Include '0' to make summary image of the entire field, including all sources. Run with '-1' for all sources and summary images.
output_image_file_type	string	png, jpg, pdf, svg	png	Output image file type. Any file type accepted by plt.savefig() is in theory valid but it is recommended to use the default.
spec_full_range	bool	True, False	True	If True, allow the spectrum with noise to be plotted over the full spectral range of the original cube.
syn_beam_dimension	list			Synthesized beam dimensions. If the primary header of the FITS files do not contain the beam information, this can be provided by the user. Accepts 1 to 3 values in order [bmaj, bmin, bpa].
channel_width	float			Channel width. This is only necessary if a source cubelet is not available, for example if user only has a mom0. Channel width must then be provided in the native units of the original data cube (typically Hz or m/s.)
min_size	int, float		0.05	Minimum image size (ARCMIN). Images will be square. Default in SIP is 6 arcmin.
snr_range	list		[2.0, 3.0]	Specify the SNR range within which to plot the lowest contour. Default in SIP is [2.0, 3.0].
survey_list	list		'none'	List of surveys on which to overlaycontours. Only the first entry will be used in the combined image if '-m' option is used. If 'none', work in offline mode. Default in SIP is 'DSS2 Blue'.
combo	bool	True, False	False	Make a combined image using ImageMagick. If a path is provided after this option, it is assumed to be the path to the 'convert' executable of ImageMagick.
user_image	list			Allow to provide an image for overlaying contours. Can use this in combination with '-s' and a list of surveys.

Continued on next page

Parameter	Type	Values	Default	Description
percentile_range	list		[10., 99.]	Percentile range when displaying the user supplied image. Requires two values. Default in SIP is [10., 99.].
spec_line	string, list		'CO(1-0)'	Allow to specify a spectral line for all sources in catalog. Also possible is 'CO(1-0)' up to (3-2) and 'OH_1667' and the 3 other OH lines which may fall within L-band observations. A second possibility is specifying parameters that are [molecule,rest frequency in GHz,label] such as ['CO',115,'CO(1-0)']. A line table is provided with the software in `SoFiA-image-pipeline/src/data/tier1_lines_reformat.list`. Default in SIP is 'HI'.
no_source_id	bool	True, False	False	Do not include the source id number in the title of the plots.
channel_maps	bool	True, False	False	Make channel maps for each source. These will be written to a multi-page pdf file in 4x5 portrait format.
spec_only	bool	True, False	False	Only make spectra, no spatial images. May be useful for multi-line sources where the moment maps are less meaningful.
plot_units	bool	True, False	False	Plot the units of the moment 0 map in Jy/beam km/s. Only works when HI line is not requested, otherwise defaults to cm^{-2} or native units.
overwrite	bool	True, False	False	Overwrite option. Will delete existing plots of the same name, if they exist before making new one.

6 Input data

ESPADA relies on SoFiA 2 as its core processing engine and therefore inherits its data format requirements. Currently, SoFiA 2 only supports the Flexible Image Transport System (FITS) format (Pence et al., 2010), and all input and output image data files must be standard FITS files with a single HDU containing the image or data cube to be processed.

ESPADA supports up to a maximum of 4 files per dataset: data cube, primary beam cube, mask cube and continuum cube with the data cube being the only mandatory input.

In standard use, both data cube and primary beam cube are expected to be three-dimensional, with the first two axes corresponding to spatial coordinates (e.g. equatorial or Galactic) and the third axis representing the spectral dimension (frequency or velocity). Two-dimensional images are also supported and are internally treated as cubes with a single spectral axis. Similarly, four-dimensional inputs are accepted provided that the fourth axis has size one (e.g. a single Stokes component), in which case it is automatically discarded.

Regarding the mask, it must have the same dimensions as the previous inputs, its values must be of integer type, and it must contain non-zero values for pixels belonging to sources. For further details

on input format and requirements, see Section 2 of the official SoFiA documentation[8].

The input data must be provided either through the parameters `input_data_set` or `input_file` within the `config.yaml` file, by using the TAP service, or by specifying a value for `input_file` with the argument `-cp|-config-parameters` through terminal.

Through the `input_data_set` parameter, the paths to the data files must be provided in a fixed order: data cube, primary beam (optional), mask (optional), and continuum (optional). These can be provided in three different formats: `<string>`, `list`, or `dictionary`.

- As `<string>`: This is intended to provide a single dataset and supports up to a maximum of 3 values. Any values beyond the third will be ignored. Examples:

```
input_data_set : data.fits pb.fits mask.fits cont.fits
input_data_set : data.fits, pb.fits, mask, cont.fits
input_data_set : data.fits, pb.fits mask cont.fits
input_data_set : data.fits, "" mask
input_data_set : data.fits, "", "" cont
input_data_set : data.fits "", mask.fits ""
input_data_set : data.fits, "" ""
```

- As `<list>` : This is intended to provide a single dataset and enforces a strict maximum of three elements in the list. If more elements are provided, ESPADA will not proceed. Examples:

```
input_data_set : [data.fits, pb.fits, mask.fits, cont.fits]
input_data_set : [data.fits, "", mask.fits ""]
input_data_set : [data.fits, pb.fits, "" cont.fits]
input_data_set : [data.fits]
input_data_set : [data.fits, "", "", ""]
```

- As `<dictionary>`: Intended to provide multiple datasets without limit. Each dataset must be identified by a key-value pair in the format `key : value`. The name assigned to the keys is irrelevant. Examples:

```
input_data_set:
  1 : [data.fits, pb.fits, mask.fits, cont.fits]
  2 : [data.fits, "", "" , cont.fits]
  3 : data.fits pb.fits mask.fits
  4 : data.fits

input_data_set: {
  1 : data.fits pb.fits, mask.fits
  2 : [data.fits, "", "", cont.fits]
  3 : data.fits, pb.fits, mask.fits, ""
  4 : data.fits
}
```

Through `input_file`, the PATH to a text file containing each dataset must be provided. The required format for each line within the file is the same as for `input_data_set`, with the exception of list usage, '[ ]'. This is not allowed, as when reading the file, the pipeline will not interpret it as a Python 'list', but rather as part of the filename, which will result in an error. Examples:

```
input_file : /home/ESPADA/dataset_file

Inside /home/ESPADA/dataset_file:
1 : data.fits pb.fits mask.fits cont.fits
```

[8] https://gitlab.com/SoFiA-Admin/SoFiA-2/-/wikis/home

```
    2 : data1.fits pb1.fits mask1.fits cont1.fits
    3 : data2.fits pb2.fits mask2.fits cont2.fits
```

If datasets are retrieved from the ASA using the TAP service, the data files are automatically structured in the correct format. It should be noted that, whenever available, the continuum file cubes will be only used during SIP execution, if applicable.

Finally, it is worth mentioning the functionality of the `use_mask` parameter, which control the use of the mask cubes. This allow to be disable cubes even if they are specified in any of the formats indicated above or are downloaded from ASA, providing extra flexibility.

7 LOGGER module - Output messages and logging

ESPADA produces a rich set of output messages to keep the user informed about pipeline progress, module execution status, and any issues encountered during the run. All messages are simultaneously written to the terminal (standard output) and to a log file for later inspection. The logging system uses colour coding to improve readability: informational messages appear in green, warnings in orange, errors in red, and critical errors in bold red. In addition, ESPADA defines a custom RAW logging level, which writes plain, unformatted messages (e.g., external software outputs) directly to the log file without terminal colour codes.

The verbosity of the output can be controlled via the verbose parameter in `config.yaml`. When set to `True` (default), the pipeline displays the full execution logs of SoFiA and SIP as they are generated.

Log files are stored in the directory specified by the log_file parameter in `config.yaml` (default: `log_dir/espada.log`), relative to output_dir. Each execution produces two log files:

- A raw log (`raw\_espada\_<date>_<time>.log`), which contains all interleaved messages exactly as generated by the parallel workers.
- A sorted log (`espada\_<date>\_<time>.log`), in which the raw log is reorganised to group messages by process ID (PID) and dataset, providing a cleaner, more readable overview of the execution.

The `clear_logs` parameter, when set to `True` (default), deletes all existing .log files in the log directory before a new run, preventing accumulation of outdated logs. Otherwise, existing logs from previous runs can be retained by setting `clear_logs: False`.

8 Execution workflow and parallelization - ESPADA

As introduced in Section 1, the ESPADA module is the central orchestrator of the ESPADA pipeline. It parses command-line arguments, loads and validates the configuration, sets up the multi-process logging environment, optionally triggers data downloads, executes the science modules (SOPAR, SIPARGS, GROUP) in parallel over all supplied datasets, performs quality assessment, reorganises the interleaved log files, and finally generates human- and machine-readable reports. For each execution, the ESPADA workflow proceeds as follows through ESPADA (see Fig. 1):

- **Parsing and validation** - The pipeline begins by parsing all arguments introduced through the terminal. If no `config.yaml`-like file is provided via the argument `"-c|--config-file"`, a default one will be used. Immediately afterwards, the CONFIG module verifies that the main configuration file contains all required parameters and valid paths necessary for proper execution. The configuration file includes specification of whether the pipeline should be run with parallelisation enabled, and how many cores should be used.

- **Logger initiation** - The pipeline then initializes the logger using the LOGGER module, based on the logger name provided in the main configuration file.

- **Data selection** - The pipeline checks if the requested ALMA data already exists on disk. If not it proceeds with a data download from the ASA, via the TAP service based on the parameters specified in `download_par.yaml` (see sections 5.2 and 9).

  If the TAP service option is disabled, the user must provide the path to the desired datasets on disk. These can be defined directly in the main configuration file or through an external file whose path must be specified in the main configuration file as described in section 6

- **Parallelization** - The pipeline performs dynamic cores allocation to process each data set. The number of parallel workers is calculated dynamically. If the host machine does not meet the minimum requirements for a dataset, the number of workers is set to one, and a warning is issued indicating that unexpected behavior might occur. More detail are provide in section 8.1.

- **Source finding (optional)** – The pipeline performs spectral line source finding by executing SoFiA based on the parameters specified in `sofia_abs_default.par` and `sofia_emi_default.par` (see Section 10). The search mode is determined by the `run_mode` parameter in `config.yaml`, which can be set to *emission*, *absorption*, or *both*. Source finding is done on a per-image-cube basis based on the noise properties of the cube. The outcome is independent of the spectral line detected.

- **Quality Assurance I** - After SoFiA is run, initial QA products are generated by the pipeline include a mom8 map of the data cube, and a comparison of the 2D projection of the SoFiA mask and the ASA clean mask, if one exists in the archive. The following quantities are also extracted from the SoFiA catalogs: Mean, Std Dev, Skewness, Kurtosis, as well as the number of sources detected (see sections 13 and 14)).

- **Visualization (optional)** - The pipeline generates publication quality figures from the SoFiA output by executing SIP based on the parameters specified in `sip_args.yaml`. If SoFiA was disabled, but SIP is enabled, then the pipeline will attempt to predict the expected source catalog name and SoFiA output based on the input data file names provided or generated at the beginning of the pipeline, assuming that SoFiA had been run earlier. More detail are provide in section 11.

- **Grouping sources (optional)** – The pipeline performs post-processing of the SoFiA detections to merge overlapping sources into composite sources through the GROUP module. Overlapping source pairs are identified by comparing their 2D footprints using a user-defined criterion (area, flux, or absolute flux) and threshold, both specified in `config.yaml` (see Section 12). These pairs are organised into groups via network analysis, and a new mask is generated in which grouped sources share a common label while isolated sources are discarded. If a valid grouped mask is produced, both SoFiA and SIP are re-run to generate the corresponding cubelets and figures exclusively for the grouped detections. See section 12 for more details.

- **Log finalization** - Once execution is complete, the logs from all (parallel) processes are organized. A final .log file is generated by merging the unsorted raw log output and the individual log files from SoFiA and SIP. These logs are structured chronologically and grouped by dataset to ensure clarity.

- **Quality Assurance II** - HTML- and JSON-based QA documentation is generated trough the module REPORT. These summarize the execution results and display key outputs from the pipeline in a visual and interactive format for quality assurance (see sections 13 and 14).

8.1 Parallelization

ESPADA distributes work over two levels: parallelism at the pipeline level, using one Python worker process per dataset, and parallelism inside each SoFiA run, controlled by the `pipeline.threads` parameter within the SoFiA parameter files, `sofia_abs_default.par` and `soifa_emi_default.par`.

Workers are dynamically assigned based on three constraints: available CPU cores, available system memory, and the number and size of the input datasets.

The process first determines the number of available CPUs. If the user specifies `num_cores`, this value is validated against the physical CPU count. When the requested value exceeds the available cores in the device, it is capped and reduced by an efficiency factor (70%) to avoid CPU saturation. Otherwise, the user-defined limit is used directly. If no value is provided, all available cores are considered. The number of concurrent workers is then computed using a dual constraint: CPU-bound limit, at most one worker per available core and Memory-bound limit, estimated from the average dataset size and available RAM. Memory usage per worker is approximated as:

$$\text{memory} = 2.25 \times \text{datasize} + 1\,\text{GB} \tag{8.1}$$

Where 2.25 is the heuristic estimate of memory consumption during SoFiA execution, according to its documentation. The final number of workers is:

$$\mathrm{W}_{max} = \min\left(\mathrm{C}_{CPU},\ \mathrm{C}_{memory},\ D\right) \tag{8.2}$$

This ensures that the pipeline does not overload CPU or RAM resources. Once the number of workers is fixed, CPU resources are allocated between: ESPADA-level parallelism and SoFiA-level parallelism. Each worker must use one core with the Python process that manages it, and the remaining available cores are divided equally among the workers for the number of threads used in SoFiA.

$$\mathrm{T}_{per\_worker} = \left\lfloor \frac{\mathrm{C}_{remaining}}{\mathrm{W}_{max}} \right\rfloor \tag{8.3}$$

where $\mathrm{C}_{remaining}$ is the number of cores not used by the worker processes themselves. Following SoFiA recommendations, the number of threads per worker is capped at 8, as performance gains beyond this point are typically negligible.

9 DATAP module - Querying and downloading data from the ALMA Science Archive

If the TAP service is enabled in the `config.yml`, the DATAP module is responsible for handling data downloads from the ASA. Part of the code has been directly adapted from the ALminer software (Ahmadi and Hacar, 2023), integrating it into ESPADA's modular structure. We take advantage of VO services for programmatic data discovery and access provided by the ASA through pyvo[9], which is an affiliated astropy package. Requesting data from the archive is done via a query, which uses the ADQL language[10]. Since this language is not commonly used within the community, several predefined query types have been implemented to simplify its usage.

We specifically note that the pipeline allows a user to enter their ALMA Science Portal user credentials and search for data that is public, proprietary data, or both, through the `credentials` and `public` parameters, respectively, in the `download_par.yaml` file. The pipeline then uses astroquery to login with credentials to the ALMA Science Portal[11]. Credentials can be retained between runs using

[9] https://pyvo.readthedocs.io/en/latest/
[10] https://www.ivoa.net/documents/latest/ADQL.html
[11] https://astroquery.readthedocs.io/en/latest/api/astroquery.alma.AlmaClass.html

the stored_credentials parameter, but it should be noted that once saved, they will remain loaded for a while even if this parameter is disabled in subsequent runs.

Each TAP service query is defined by two essential elements: (1) the query_type parameter, and (2) the parameters required for the query, provided under query_par. Within query_par, there are two categories of parameters: common parameters for all queries, and those specific to the selected query type. All available parameters are included within the download_par.yaml file, and commented out as necessary (see Table 2). The predefined query types currently supported are:

- 'proposal' - specify the data by the proposal_id;
- 'conesearch' - search a radius around a specified ra and dec;
- 'target' - search a radius around a target name;
- 'keysearch' - search the archive based on a dictionary of keywords;
- 'free' - perform an ADQL query and return the results.

In all cases, these searches can be further narrowed by specifying the filename_must_include parameter with a list of strings, e.g. filename_must_include : ['spw25'].

In addition download_par.yaml also includes download_par parameters to handle individual data file types retrieved via the query, which modify the behavior of the pipeline. These are remove_compressed_file, and remove_archive_mask to control whether to delete the original primary beam and mask files after processing. These files are not used in their downloaded form within the pipeline. In the case of the primary beam, it is stored compressed within the ASA and it must be extracted by the DATAP module before source finding. For the mask, the original file contains floating-point values, which are incompatible with SoFiA; therefore, a copy of the file is created with integer-type data. It may be common for a user to run ESPADA multiple times on the same datasets while modifying SoFiA and SIP parameters to explore different outcomes. If the deletion parameters are enabled, the compressed files will be downloaded again with each execution. However, if the files are retained, the download system will detect their presence and skip the download step, allowing the pipeline to continue without re-downloading previously obtained files.

It worth to mention that ESPADA automatically generates a file named espada_input_file.txt inside the main output directory (output_dir). This file lists every downloaded dataset (cube, primary beam, mask, and continuum) using the same format expected by the input_file parameter (see examples above). The file is overwritten on each new TAP-based execution, always reflecting the most recent download. To re-run the pipeline on the same datasets without having to go through the ALMA archive again, simply set enable_tap_service: false in config.yaml or through the terminal and point the input_file parameter to the generated file.

All parameters are described in detail in Table 2 and, in a more concise form, within the download_par.yaml file. Below are several additional examples of QUERY beyond those already included in the file:

```
query_type: 'proposal'
query_par:
  proposal_id: '2016.1.00778.S'
  point: False
  public: True
  published: None
  print_targets: True
  print_query: True
```

```
query_type: 'target'
query_par:
```

```
    sources: ['V605 Aql']
    search_radius: 2.
    point: False
    public: True
    published: None
    print_targets: True
    print_query: True
```

```
  query_type: 'target'
  query_par:
    sources: ['V605 Aql']
    search_radius: 2.
    point: False
    public: True
    published: None
    print_targets: True
    print_query: True
```

```
  query_type: 'free'
  query_par:
    query_str: *"SELECT * FROM ivoa.obscore WHERE ((LOWER(
       proposal_abstract) LIKE '%planet-forming disk%')) AND (
       spatial_resolution < 0.5) AND (LOWER(data_rights) LIKE '%public%')
       AND (LOWER(scan_intent) LIKE '%target%') ORDER BY proposal_id"*
      point: False
      public: True
      published: None
      print_targets: True
      print_query: True
```

10 SOPAR module - Source finding using SoFiA

The SOPAR module is responsible for running the stand-alone SoFiA source finding software on the image cubes. SoFiA has been rigorously tested and is widely used in the HI extragalactic community, and the software is described in a series of papers by Serra et al. (2015) and Westmeier et al. (2021), so we do not go into further detail here. Further information is also available on the SoFiA wiki[12] and in the user manual which is available at the same site.

Whether SoFiA is run in *absorption*, *emission*, or *both* mode is controlled by the `run_mode` parameter in the `config.yaml`. If `enable_sofia : False`, the pipeline will skip this step, and proceed to the SIP module, assuming SoFiA has already been run on the data, and expecting the pipeline file structure for SIP as would have followed based on the names of the files generated from the DATAP query, or based on the datasets specified in the `input_data_set` or `input_file` parameters `config.yaml` (see section 6 for more details).

The SoFiA parameter files for *absorption* and *emission* are specified separately in the `sofia_abs_file` and `sofia_emi_file` parameters, respectively, in the `config.yaml` file. ESPADA is packaged with default parameter files that follow the SoFiA template, but a user can also modify these or provide their own. There is a series of parameters that cannot be changed, and their value is determined or changed internally to ensure the correct functioning and behavior of ESPADA throughout the entire execution. Below are the parameters that cannot be changed or will be directly ignored if an attempt is made to modify them:

[12] https://gitlab.com/SoFiA-Admin/SoFiA-2/-/wikis/home

- `input.data, input.primaryBeam, input.mask` : These three parameters are controlled by the native ESPADA parameters, `input_data_set`, `input_file` within the `config.yaml` or by the DATAP module if downloading data using the TAP service.
- `input.invert` : To control the type of source being searched for, the parameter `run_mode` should be used with its modes *emission, absorption* and *both*.
- `pipeline.threads`: ESPADA carries out a main parallelization within the main module ESPADA and a secondary one corresponding to the SoFiA software. To avoid possible errors, the calculation of this parameter is internally optimized based on the parameter `num_cores`. The possible values will always range between 1 and 8, which is the maximum value recommended in the SoFiA documentation. For more details, see section 8.
- `scfind.enable`, `contsub.enable`, `scaleNoise.enable`, `background.enable`, `threshold.enable`, `reliability.enable` and `dilation.enable`: These parameters will be automatically set to **False** if a mask is used in the SoFiA execution. Note that the use of the mask can be deactivated even if it is available, using the parameter `use_mask`. They can be modified except when the mask is active.
- `output.directory`: The output directory is controlled by the parameter `output_dir` in the `config.yaml`. For more details on the directory and file structure, see section 15.
- `output.filename`: ESPADA maintains partial control over file names to avoid confusion between execution modes and different datasets. It can be modified, but the indicated value will include additional text necessary for file differentiation. For more details on file and directory naming conventions, see section 15.
- `output.writeCatXML`: The catalogs are necessary to run SIP in the next step of the ESPADA workflow (see Fig. 1), so it has been forced to always create the catalog in XML format, even if SIP is disabled, since it contains metadata that can potentially be used during ESPADA execution.
- `output.writeCubelets`: The datacubes obtained from the SoFiA execution are necessary to run SIP in the next step of the ESPADA workflow (see Fig. 1), so it has been forced to always create them, even if SIP is disabled.

In addition to the above parameters, the parameters `output.dataFormat`, `output.writeCatASCII` and `output.overwrite`, although they can be modified (disabled), are recommended to be kept to their default settings.

The pipeline configuration file, `config.yaml`, also contain the `abs_flag_cube` and `auto_setup` parameters, which also control the parameters used in SoFiA. The first is used when the `run_mode` parameter is set to *both* and allows the user to use or not use the absorption sources found as masks in the next run (emission). The second, still in an experimental phase, sets the SoFiA parameter file values based on the properties listed in the header of the image cubes themselves – for example, choosing smoothing kernels. The parameters that are modified in this case are: `scfind_kernelsXY`, `linker_minSizeXY` y `filter_minSNR`. **red**We have an internal memo which identifies the potential SoFiA parameters and a proposal of how to set them based on this information. This document is include in Appendix B.

11 SIPARGS module - producing images with SIP

The SIP module is responsible for running SIP on the (expected) SoFiA outputs on the SOPAR module. Within the `config.yaml` file, if `enable_sofia: False`, and `enable_sip: True`, the pipeline will skip the SOFIA module, but will attempt to run SIP by predicting the filenames from DATAP, or as

specified in the INPUT DATA section of `config.yaml`. If `enable_sip: False` the pipeline will skip this step and proceed with the final steps specified by the main ESPADA module.

As a stand-alone program SIP runs on the command line and takes command line inputs. To interface between with these command line inputs, ESPADA is packaged with a default `sip_args.yaml` file.

SIP generates png images per source to visualize the results of SoFiA. For the case where SoFiA detects multiple sources in the same image cube, we have adapted SIP to also make overview images containing all the sources found within the image cube.

11.1 Limitations of using SIP in isolation for *both* mode

The primary (i.e., required) input is a SoFiA-generated source catalog, provided either in ASCII or XML format. SIP then locates the corresponding SoFiA outputs by following the standard SoFiA file structure. As mentioned earlier, SIP can be executed even with the source finding SOPAR module disabled, provided that this module has been run previously and has produced the required catalogs for the specified mode and datasets.

If no prior execution has been performed, or if it was carried out in directories different from the current one, SIP can still be run, but with certain limitations. ESPADA is designed to operate in parallel across multiple datasets and in different modes, depending on the `run_mode` parameter (see Section 5.1). However, at present there is no way to specify, via the `sip_args.yaml` file, all the required catalogs when ESPADA is executed in *both* mode.

The `catalog_file` parameter accepts only a list as input, where each element must be the PATH to a catalog file corresponding to each dataset defined in the `input data` section, and in the same order. This pproach is valid **exclusiveky** for the *emission* and *absorption* modes. In *both* mode, this parameter has a critical limitation: it is not possible to specify separate catalogs for the emission and absorption runs. ESPADA will still execute, but it will use the same catalog for both modes, resulting in duplicated values in the outputs and an incorrect SIP execution for at least one of the two modes. **Therefore, manually providing catalogs via `catalog_file` when `run_mode` is set to `both` is not recommended under any circumstances.**

Similarly, this approach is discouraged when using TAP service. In such case, datasets may be internally ordered differently between executions, which can cause the catalogs specified in `sip_args.yaml` to be applied to incorrect datasets, potentially leading to errors or even causing ESPADA to crash.

12 GROUP module - Grouping source detections

The module GROUP is responsible for the post-processing of the source detections obtained with SoFiA in order to merge spatially-overlapping sources into a single source. Inside the `config.yaml` file, if the `enable_group` parameter is `False`, the pipeline will skip this module.

The workflow inside GROUP is based on 3 sequential stages: mask retrieval, overlap analysis of source pairs, and grouping sources.

First, the 3D detection mask generated by SoFiA is needed. This mask is retrieved from the current execution, if available; otherwise, the process attempts to retrieve it from previous executions using the file naming convention. If no valid mask is found, the grouping stage is skipped.

Each detected source is characterized by its mask and the associated data cube. For each source i we use mask and cube to derive the integrated intensity map (mom0 image) and the 2D footprint of the mask, respectively. From these, we measure total area (i.e., number of spatial pixels in the 2D footprint), total flux (sum of the value of the detected pixels), and total absolute flux (sum of the absolute value of the detected pixels). Next, pairwise overlaps between sources are evaluated using these quantities. For a given pair (i,j), the overlap region is defined as the intersection of their 2D

footprints. From this, fractional overlap metrics are calculated:

$$f_i^{(X)} = \frac{\mathrm{X}_{overlap,i}}{\mathrm{X}_{total,i}}, \quad f_j^{(X)} = \frac{\mathrm{X}_{overlap,j}}{\mathrm{X}_{total,j}} \tag{12.1}$$

where X is set by the `overlap_mode` parameter and can be *area*, *flux* or *absflux*. A pair of sources is considered overlapping only if both fractions exceed a user-defined threshold via the `overlap_threshold` parameter:

$$f_i^{(X)} > T \quad \text{and} \quad f_j^{(X)} > T \tag{12.2}$$

where T is the threshold value between 0 and 1. This symmetric condition prevents asymmetric associations driven by large/small source biases.

Overlapping pairs are interpreted as edges in an undirected graph, where nodes represent source IDs. Groups are then defined as connected components of this graph. This ensures transitive grouping, i.e., if A overlaps with B and B overlaps with C, all three sources are merged even if A and C do not directly overlap. A new detection mask is constructed from the identified groups. For each group, all constituent sources are assigned a common label (chosen as the minimum source identifier within the group), while sources not belonging to any group are removed. The resulting mask therefore contains only grouped detections. For ungrouped sources, all products created prior to the grouping algorithm are of course retained and are part of the final result of the ESPADA run.

Finally, if a valid grouped source mask is produced, both SoFiA and SIP are run again in order to produce the corresponding cubelets and images only for these new sources. To ensure that SoFiA does not try to search for new sources in this run, the parameters `scfind.enable`, `linker.enable` and `reliability.enable` are set to "false".

Our experience thus far is that X = *absflux* and $T = 0.8$ work well for point sources, which are the target of this algorithm (FYR). This conclusion is based on a limited number of tests performed by injecting point sources into real cubes. A larger, systematic testing campaign would be required in order to establish the range of point-source SNR over which this choice of X, T gives good results.

In contrast, we note that there is no objective way to define an optimal X, T for resolved sources. The reason is that different lines are likely to have intrinsically different resolved morphology and, therefore, an intrinsically partial overlap. Therefore, a lower T might be acceptable than for point sources – with the exact choice being guided by science goals.

13 REPORT module - Generation of an execution final reports

The REPORT module is responsible for compiling all the information generated during a pipeline run into both human- and machine-readable formats. It is invoked automatically at the end of every execution, provided that the `make_report` parameter in `config.yaml` is set to `True`.

The report generation process is structured in three stages: data parsing, image preparation, and output rendering.

The module first processes the structured results returned by each worker. These worker results contain, for every dataset, the outputs of all active modules, SOPAR, SIPARGS, GROUP and additionally the quality assurance (see section 14). The parser extracts and organize the following information into a unified hierarchical structure:

- **Pipeline metadata**: run identifier, version, start and end times, total duration, and system environment (Python version, hostname, working directory).
- **Execution statistics**: total number of workers, number of successful and failed workers, and aggregate counts of warnings and errors (still under development).

- **Per-dataset results**: for each input dataset, the software executed, its mode (*emission* / *absorption*), process ID, execution status, error messages, command line used, any modifications made to the SoFiA parameter file, the full content of the associated log files, and a complete catalogue of all output images and data files.
- **Global summary**: totals for datasets processed, datasets with errors or warnings, and the overall number of images and files generated.

Because some output formats (notably Encapsulated PostScript, EPS) are not suitable for web display, the REPORT module includes an automatic image conversion step. EPS files generated by SoFiA (e.g., diagnostic and reliability plots) are converted to PNG format using Ghostscript[13]. All images are then copied into a dedicated `images/` subdirectory within the report folder, and their paths are updated accordingly in the internal data structure. If this software is unavailable, the images simply will not be displayed, but the original images will still be available in the same directory.

The parsed and prepared data are used to generate two complementary outputs:

- **JSON report** (`report.json`): a complete, machine-readable dump of the entire parsed structure, including the full content of both the raw and the reorganised log files. This format is intended for programmatic access, integration with databases, or downstream analysis tools.
- **HTML report** (`index.html`): an interactive, self-contained web page rendered from a Jinja2 template. The HTML report presents a curated subset of the information: an execution summary, the configuration used (with human-readable parameter names and visual indicators for boolean values), per-dataset galleries of moment maps, spectra, position–velocity diagrams, diagnostic plots, and quality-assessment comparisons. The reorganised execution log is embedded for direct inspection.

Both reports are written to a time-stamped subdirectory within the main output folder, allowing multiple pipeline runs to be documented and compared without overwriting previous results.

14 Quality Assurance of the pipeline products

ESPADA incorporates a multi-stage quality assurance (QA) workflow designed to help the user rapidly assess the reliability of the source-finding results and the overall quality of the data products. QA is performed automatically after each SoFiA execution and is complemented by the final interactive HTML report.

14.1 Stage I – Per-dataset diagnostics

Immediately after SoFiA completes, the pipeline generates a set of diagnostic products for each dataset and each execution mode (emission/absorption):

- **Moment 8 image**: a two-dimensional projection of the data cube obtained by taking the maximum (emission) or minimum (absorption) value along the spectral axis for each spatial pixel. When a primary-beam cube is available, it is applied to the data before projection. The resulting image provides a quick visual overview of the line emission or absorption across the entire field of view.
- **Mask comparison**: if an external mask is available—either downloaded from the ALMA Science Archive (ASA) or provided by the user—the pipeline produces a side-by-side comparison figure

[13] https://www.ghostscript.com/

showing the moment 8 image, the 2D projection of the SoFiA detection mask, and the 2D projection of the external mask. This allows a direct visual evaluation of whether SoFiA is recovering the expected sources.

- **Cube noise statistics**: the pipeline extracts from the SoFiA XML catalogue the mean, standard deviation, skewness, and kurtosis of the noise distribution, as well as the total number of detected sources. These metadata provide an at-a-glance summary of the cube quality and are particularly useful for comparing multiple datasets. These quantities are recorded both in the log and in a dedicated statistics file (`*_comparison_stats.txt`).

All diagnostic images and statistics files are saved under a `quality_assessment_products/` subdirectory within each dataset's output folder.

14.2 Stage II – Interactive HTML report

The second QA layer is the self-contained HTML report generated by the REPORT module (see Section 13). Accessible through any web browser, the report is organised into three main tabs:

- **Dataset list**: an expandable, colour-coded panel for each dataset. The colour indicates the global execution status (green: success; orange: warnings; red: errors; purple: module disabled). Inside each panel, individual software executions (SoFiA-2, SIP) are displayed with their own status indicator, the number of warnings, and buttons to inspect the full log or the parameter file / command used.
- **Quality assessment gallery**: embedded directly within each dataset panel, the QA section displays:
  - The cube noise statistics in tabular form.
  - The moment 8 image and, when available, the mask comparison figures.
- **SoFiA-2 and SIP galleries**: the report organises all generated images—diagnostic plots, reliability plots, moment maps (0, 1, 2), spectra, position–velocity diagrams, and survey overlays—grouped by software, execution mode (absorption/emission), and processing stage (regular/group). SIP outputs are further collapsible by source ID, allowing easy navigation even for fields with many detections.
- **ESPADA Log tab**: provides full access to both the raw (chronological) and the reorganised (by process ID) execution logs, enabling detailed forensic examination of the pipeline run.
- **Configuration tab**: displays the complete `config.yaml` and, if applicable, `download_par.yaml` parameters in a human-readable format, with boolean values rendered as coloured indicators (Enabled / Disabled) and complex structures (lists, dictionaries) shown in expandable detail views.

The HTML report is entirely self-contained: all images are embedded by relative path, and the necessary CSS and JavaScript resources are copied into the report directory. This design allows the report folder to be shared as a single archive, providing collaborators with a complete, interactive record of the pipeline execution and its scientific outputs.

15 Folder/Directory structure of pipeline outputs

A run of ESPADA creates a well-organized output hierarchy rooted in the directory specified by the `output_dir` parameter in `config.yaml` (default: `espada\_run/`). If left blank, the default directory

will be created in the current working directory. The naming of files and subdirectories follows a consistent pattern that incorporates the input dataset name and the execution mode (*emission* or *absorption*), making it easy to trace each product back to its origin.

For a given input data cube named `<asa\_filename>.fits` (e.g., UID123456789_cube.I.pbcor.fits), ESPADA uses the root name, `<asa\_filename>`, as the base identifier for all associated outputs. If multiple FITS cubes are processed in a pipeline execution then the folders `espada_<asa_filename>` will be produced for each cube and contain the corresponding outputs.

Within `output\_dir`, ESPADA creates the following top-level directories:

- `archive\_data/` (optional): Contains data downloaded from the ALMA archive when `enable_tap_service` is set to True. This includes the original FITS cubes, primary beams, masks, continuum images, and any compressed (.gz) files that were extracted. This may change subject to change depending on the `data_dir` parameter within the `download_par.yaml` file.
- `log\_dir/`: Stores log files. The final sorted log is named espada_<date>_<time>.log, while the raw unsorted log is prefixed with 'raw_'. This folder is also subject to change depending on the `log_file` parameter within the `config.yaml` file.
- `espada\_<asa\_filename>/`: The main output directory for a given dataset. All SoFiA and SIP products, as well as quality assessment files for that cube, are placed here.
- `report\_<date>\_<time>/`: Contains the HTML and JSON reports summarizing the entire pipeline execution, including metadata, configuration, and links to key output products. If no other report directory exists, this one will simply appear without the timestamp.

Inside `espada\_<asa\_filename>/`, ESPADA distinguishes between emission and absorption processing modes by prefixing all filenames and subdirectory names with either `'emission\_'` or `'absorption\_'`. This dual structure appears regardless of whether `run_mode` is set to *emission*, *absorption*, or *both*. The internal structure for each data cube is as follows:

- `<mode>\_<asa\_filename>\_cubelets/`: Contains per-source cutouts (cubelets) and individual source catalog files (TXT or XML) generated by SoFiA.
- `<mode>\_<asa\_filename>\_figures/`: Contains per-source plots (moment maps, spectra, position-velocity diagrams) generated by SIP.
- `<mode>\_<asa\_filename>\_logfile.log`: The raw log output from SoFiA for that specific run.
- `<mode>\_<asa\_filename>\_sip.log`: The log output from SIP for that specific run.
- `<mode>\_<asa\_filename>\_diagnostic.eps,\_rel.eps,\_skellam.eps`: Diagnostic plots produced by SoFiA.
- `<mode>\_<asa\_filename>\_cat.txtand\_cat.xml`: Source catalogs in ASCII and XML formats.
- `<mode>\_<asa\_filename>\_mask.fits`: The 3D source mask produced by SoFiA.
- `<mode>\_<asa\_filename>\_mask-2d.fits`: The 2D projection of the source mask.
- `<mode>\_<asa\_filename>\_mom0.fits,\_mom1.fits,\_mom2.fits,\_chan.fits`: Moment maps and channel maps.
- `<mode>\_<asa\_filename>\_mom0.png,\_sources.png`, etc.: PNG versions of moment maps and source identification images.

Withininally, `espada\_<asa\_filename>/`, ESPADA creates a `quality\_assessment\_products/` folder when make_report is enabled. This folder contains:

- `<asa\_filename>\_QA.png`: A composite image overlaying the moment-8 projection (max/min along the spectral axis), the SoFiA 2D mask, and the user-provided or archive-downloaded mask for visual comparison.
- `<asa\_filename>\_comparison\_stats.txt`: A text file with quantitative statistics comparing the SoFiA-identified mask with the provided mask, including pixel overlap fractions, flux fractions, and per-source metrics when applicable.

Finally the `report\_<data>\_<time>` directory contains an entry point index.html for browsing the pipeline results in a web browser, along with report.json which stores all metadata, configuration, and output file references in machine-readable format. The `images/` subfolder holds embedded images used in the report, and the `resources/` folder contains CSS stylesheets for the HTML layout. See all the details in Section 13.

The directory tree below illustrates the complete output structure generated by ESPADA after a typical execution:

```
<working directory>
├── data_archive
│   ├── <asa_filename>_pbcor.fits
│   ├── <asa_filename>_pb.fits
│   ├── <asa_filename>_mask.fits
│   └── <asa_filename>_.mfs.I.pbcor.fits .3 <pickle files>.pickle
├── log_dir
│   ├── espada_<data>_<time>.log
│   └── raw_espada_<data>_<time>.log
├── espada_<asa_filename>
│   ├── absorption_<asa_filename>_cubelets
│   │   └── <fits, txt output from SoFiA per source>
│   ├── absorption_<asa_filename>_figures
│   │   └── <png, txt output from SIP per source>
│   ├── absorption_<asa_filename>_logfile.log
│   ├── absorption_<asa_filename>_<diagnostic,rel,skellam>.eps
│   ├── absorption_<asa_filename>_cat.<txt,xml>
│   ├── absorption_<asa_filename>_<mask*,mom*,chan>.fits
│   ├── absorption_<asa_filename>_<mom*,sources>.png
│   ├── absorption_<asa_filename>_sip.log
│   ├── quality_assessment_products
│   │   ├── absorption_<asa_filename>_QA.png
│   │   ├── absorption_<asa_filename>_comparison_stats.txt
│   │   ├── emission_<asa_filename>_QA.png
│   │   └── emission_<asa_filename>_comparison_stats.txt
│   ├── emission_<asa_filename>_cubelets
│   │   └── <fits, txt output from SoFiA per source>
│   ├── emission_<asa_filename>_figures
│   │   └── <png, txt output from SIP per source>
│   ├── emission_<asa_filename>_logfile.log
│   ├── emission_<asa_filename>_<diagnostic,rel,skellam>.eps
│   ├── emission_<asa_filename>_cat.<txt,xml>
│   ├── emission_<asa_filename>_<mask*,mom*,chan>.fits
│   ├── emission_<asa_filename>_<mom*,sources>.png
│   ├── emission_<asa_filename>_sip.log
│   │   ├── <asa_filename>_QA.png
│   │   └── <asa_filename>_comparison_stats.txt
├── report_<data>_<time>
│   ├── images
│   ├── index.html
│   ├── report.json
│   ├── resources
│   │   ├── base.css
│   │   ├── collapsible.css
│   │   └── dataset_tab.css
```

Acknowledgements

Acknowledges support from the ESO/ALMA development study "Prototype for ALMA Spectral Line Advanced Data Product Pipeline" funded by the framework of the ESO “Advanced Study for Upgrades

of the Atacama Large Millimeter/submillimeter Array (ALMA)" (CFP/ESO/22/328/AMA)

List of acronyms

ADP advanced data product

ADQL Astronomical Data Query Language

ASA ALMA Science Archive

ESPADA Advanced Data Product ALMA Pipeline

ESPADA Advanced Data Product ALMA Pipeline

QA quality assurance

SIP SoFiA Imaging Pipeline

SoFiA Source Finding Application

TAP Table Access Protocol

VO Virtual Observatory

References

Ahmadi, Aida and Alvaro Hacar (June 2023). *ALminer: ALMA archive mining and visualization toolkit*. Astrophysics Source Code Library, record ascl:2306.025. ascl: 2306.025.

Pence, W. D. et al. (Dec. 2010). "Definition of the Flexible Image Transport System (FITS), version 3.0". In: 524, A42, A42. DOI: 10.1051/0004-6361/201015362.

Serra, Paolo et al. (Apr. 2015). "SOFIA: a flexible source finder for 3D spectral line data". In: 448.2, pp. 1922–1929. DOI: 10.1093/mnras/stv079. arXiv: 1501.03906 [astro-ph.IM].

Westmeier, T. et al. (Sept. 2021). "SOFIA 2 - an automated, parallel H I source finding pipeline for the WALLABY survey". In: 506.3, pp. 3962–3976. DOI: 10.1093/mnras/stab1881. arXiv: 2106.15789 [astro-ph.IM].

A Change the execution command of SoFiA and SIP in ESPADA

As mentioned in section 3, if you wish to change the execution commands of SoFiA and SIP within ESPADA for any reason, the steps to follow are::

- Look for the function **run_sofia** inside the SOPAR module.
- Inside each function, look for the line cmd = ["sofia", f"temp_file_path"] in each of the if blocks corresponding to each of the usage modes (see below).
- Change the (str) `"sofia"` for the corresponding command used to run SoFiA on the device.
- Look for the function **generate_command** inside the SIPAGRS module.
- Look for the line cmd = ["sofia_image_pipeline"]
- Change the (str) `"sofia_image_pipeline"` for the corresponding command used to run SIP on the device.

B SoFiA parameters calculated when `auto_setup: True`

Below is a team internal memo proposing the SoFiA parameters and how to set them in an automated way, based on a FITS cube's header information, when `auto_setup: True`.

Automatic setup options for key SoFiA user settings

Source finding (`scfind`)

Parameter	Header keywords	Suggested SoFiA defaults
`scfind.kernelsXY`	`bmaj`, `bmin`, `cdelt1`	scfind.kernelsXY = 0, *x*, 2*x* where *x* = (`bmaj` + `bmin`) / (2 × |`cdelt1`|)
`scfind.kernelsZ`	—	scfind.kernelsZ = 0, 3, 7, 15
`scfind.threshold`	—	scfind.threshold = 3.8 *Note: this requires clean data; a higher threshold in the range of 4–5 may be necessary for data with artefacts.*

Linking (`linker`)

Parameter	Header keywords	Suggested SoFiA defaults
`linker.radiusXY`	—	linker.radiusXY = 2
`linker.radiusZ`	—	linker.radiusZ = 3
`linker.minSizeXY`	`bmaj`, `bmin`, `cdelt1`	linker.minSizeXY = *x* where *x* = (`bmaj` + `bmin`) / (2 × |`cdelt1`|)
`linker.minSizeZ`	—	*No idea what default would be appropriate here; possibly no less than 3, but actual value depends on science case and nature & spectral resolution of the data.*

Reliability filtering (`reliability`)

Parameter	Header keywords	Suggested SoFiA defaults
`reliability.enable`	—	reliability.enable = true
`reliability.minSNR`	`bmaj`, `bmin`, `cdelt1`	If `bmaj` and `bmin` defined in FITS header: reliability.minSNR = 3.0 If `bmaj` or `bmin` undefined: reliability.minSNR = *x* where *x* = (3 / 2) × sqrt[π*ab* / ln(2)], *a* = beam major axis size in pixels, *b* = beam minor axis size in pixels
`reliability.threshold`	—	*Not sure what to pick here; a high value (≳ 0.9) will yield high reliability at the cost of lower completeness; a low value (≲ 0.7) would improve completeness, but at the cost of low reliability. Perhaps leave to the user to decide.*
`reliability.scaleKernel`	`naxis1`, `naxis2`, `naxis3`, `bmaj`, `bmin`, `cdelt1`, *spectral resolution*	*This one is tricky. The optimal value should scale with the cube root of the number of resolution elements in the data cube (i.e. number of beam area × spectral resolution elements in the data cube, assuming a 3D cube and 3D reliability parameter space). Setting the base value will be tricky, but we could try to pin this down to a typical ALMA cube with sufficiently large number of resolution elements.*